PARALLEL COMPUTING · OPENMP

Sắp xếp song song trong C

Cho nhiều nhân CPU cùng sắp xếp một mảng

Trình bày: [Tên người 1] · [Tên người 2]

NỘI DUNG · 10 PHÚT

Hai phần, hai người trình bày

NGƯỜI 1 · PHÚT 0–5

Odd-Even Sort

- Song song là gì? Speedup, barrier
- Biến bubble sort thành song song
- Demo: song song mà lại chậm hơn!

NGƯỜI 2 · PHÚT 5–10

Merge Sort

- Chia đôi mảng, mỗi nửa một nhân
- Demo: nhanh hơn khoảng 5 lần
- Kết luận: 3 bài học

CÂU HỎI MỞ ĐẦU

Máy có 16 nhân thì sắp xếp nhanh hơn 16 lần?

Ý tưởng song song: chia việc cho nhiều nhân cùng làm, giống 4 người cùng xếp một chồng bài thay vì 1 người.

Speedup

Thời gian chạy 1 nhân ÷ thời gian chạy nhiều nhân.
Speedup = 5 nghĩa là nhanh hơn 5 lần.

Barrier (đồng bộ)

Điểm mà mọi nhân phải chờ nhau làm xong.
Chờ nhau thì tốn thời gian.

THUẬT TOÁN 1 · ODD-EVEN SORT

Bubble sort, tách thành 2 pha xen kẽ

Bubble sort làm **lần lượt** từng cặp, nên không chia việc được.

Pha chẵn: cặp (0,1), (2,3), (4,5)...
Pha lẻ: cặp (1,2), (3,4)...

Các cặp trong cùng một pha **không đụng nhau**, nên nhiều nhân làm cùng lúc được.

Ví dụ với mảng 5 phần tử

Ban đầu	5 2 4 1 3
Pha chẵn	2 5 1 4 3
Pha lẻ	2 1 5 3 4
Pha chẵn	1 2 3 5 4
Pha lẻ	**1 2 3 4 5**

CODE C + OPENMP

Chỉ cần thêm một dòng #pragma

```
#pragma omp parallel
for (int phase = 0; phase < n; phase++) {
    #pragma omp for  // chia các cặp cho các nhân
    for (int i = phase % 2; i < n - 1; i += 2)
        if (a[i] > a[i+1]) swap(&a[i], &a[i+1]);
    // hết mỗi pha: các nhân CHỜ NHAU (barrier)
}
```

phase % 2 = 0 → pha chẵn, = 1 → pha lẻ. Biên dịch: **gcc -O2 -fopenmp**

DEMO · ./ODD_EVEN 20000

Song song mà chậm hơn ~20 lần!

1 NHÂN	16 NHÂN
0,27 s	6,26 s

20.000 pha = 20.000 lần chờ nhau, mà mỗi pha lại rất ít việc. **Thời gian chờ nhiều hơn thời gian làm.**

6 · Người 1 · số đo trên máy 16 luồng, mỗi lần chạy có thể khác

THUẬT TOÁN 2 · MERGE SORT

Chia đôi mảng, mỗi nửa giao cho một nhân



Hai nửa **độc lập** → làm cùng lúc. Xong rồi **trộn ngược lên** thành mảng đã sắp xếp.

CODE C + OPENMP

Mỗi nửa mảng là một task

```
void sort(int *a, int lo, int hi) {
  if (hi - lo < CUTOFF) {
    sort_tuan_tu(a, lo, hi); return;
  }
  int mid = (lo + hi) / 2;
  #pragma omp task
  sort(a, lo, mid);
  #pragma omp task
  sort(a, mid, hi);
  #pragma omp taskwait
  merge(a, lo, mid, hi);
}
```

taskwait

Đợi cả hai nửa xong rồi mới trộn được.

CUTOFF = 10.000

Đoạn nhỏ thì một nhân tự làm, không chia tiếp. Tránh lỗi của Odd-Even!

DEMO · ./MERGE_SORT · 10 TRIỆU SỐ

Nhanh hơn ~5 lần



Có 16 nhân... sao không nhanh 16 lần?

KẾT LUẬN

Không nhanh 16 lần vì các nhân phải chờ nhau, và bước trộn cuối chỉ một nhân làm

1

Phần độc lập

Chọn thuật toán có các phần không phụ thuộc nhau.

2

Việc đủ lớn

Mỗi phần việc phải lớn hơn chi phí chia việc.

3

Ít chờ nhau

Càng ít barrier càng tốt.

Cảm ơn! Mời các bạn đặt câu hỏi.